



WebSockets и Spring Boot

1. Введение

До сих пор наше взаимодействие было либо по запросу клиента (REST), либо между серверами (RabbitMQ). Но что, если серверу нужно *самому* инициировать отправку данных прямо в браузер пользователя? Например, обновить график в реальном времени, показать уведомление или сообщение в чате.

Традиционный HTTP работает по модели «запрос-ответ». Сервер молчит, пока его не спросят. Постоянный опрос (polling) сервера каждые 5 секунд — это неэффективно и медленно.

WebSockets решают эту проблему, создавая постоянный двунаправленный канал связи.

Хотя в больших энтерпрайз-проектах часто используют надстройки типа STOMP, понимание «чистых» веб-сокетов критически важно. Это «база», на которой строится все остальное. Это знание необходимо для написания высокопроизводительных сервисов (например, игровые сервера, финансовые тикеры), где важен каждый байт и миллисекунда, а накладные расходы STOMP излишни.

Необходимое ПО:

1. **JDK 17+, Maven.**
2. **Любой современный браузер (Chrome/Firefox).**
3. **Postman или curl для триггера событий.**

2. Теория

Протокол WebSocket и Handshake

WebSocket — это не HTTP, но он начинается с HTTP.

1. Клиент отправляет обычный GET-запрос с заголовками:
 1. Connection: Upgrade
 2. Upgrade: websocket
2. Если сервер согласен, он отвечает статусом 101 Switching Protocols.
3. С этого момента TCP-соединение остается открытым, и обмен данными идет в виде легких бинарных или текстовых фреймов, а не тяжелых HTTP-заголовков.

WebSocketHandler в Spring



| WebSockets

В Spring для «чистых» сокетов мы реализуем интерфейс `WebSocketHandler` (или наследуемся от удобного `TextWebSocketHandler`). У него есть ключевые методы:

- **`afterConnectionEstablished`** — соединение открыто, сохраняем сессию;
- **`handleTextMessage`** — пришло сообщение от клиента;
- **`afterConnectionClosed`** — клиент отключился, удаляем сессию;
- **`handleTransportError`** — ошибка сети.

Управление состоянием

В отличие от REST (где нет состояния), WebSocket **stateful**. Сервер должен помнить всех подключенных клиентов. Обычно это делается через потокобезопасную коллекцию `CopyOnWriteArrayList<WebSocketSession>` или `ConcurrentHashMap`.

`WebSocketSession` не является потокобезопасной для одновременной записи. Если вы пытаетесь отправить сообщения в одну и ту же сессию из разных потоков одновременно, вы получите ошибку. В этом занятии мы будем использовать синхронизацию или однопоточную рассылку.

3. Практический разбор

Создадим отдельный микросервис `simple-notification-service`. Он будет держать связь с браузером. У нас будет REST-эндпоинт, на который мы (как администратор) можем «стукнуть», и этот «стук» мгновенно улетит всем подключенным в браузере пользователям.

Создание проекта

Создайте новый проект на Spring Initializr:

- **Artifact:** `simple-notification-service`;
- **Dependencies:** `Spring Web`, `WebSocket`.

Реализация Handler (сердце WebSocket)

Создайте пакет `handler` и класс `NotificationWebSocketHandler`.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;
import org.springframework.web.socket.CloseStatus;
import org.springframework.web.socket.TextMessage;
import org.springframework.web.socket.WebSocketSession;
```



| WebSockets

```
import org.springframework.web.socket.handler.TextWebSocketHandler;

import java.io.IOException;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;

@Component
public class NotificationWebSocketHandler extends TextWebSocketHandler
{

    private static final Logger log =
    LoggerFactory.getLogger(NotificationWebSocketHandler.class);

    // ConcurrentHashMap.newKeySet() – потокобезопасный Set,
    // эффективнее CopyOnWriteArrayList при частых изменениях
    private final Set<WebSocketSession> sessions =
    ConcurrentHashMap.newKeySet();

    @Override
    public void afterConnectionEstablished(WebSocketSession session) {
        sessions.add(session);
        log.info("Новое подключение: id={}, всего активных: {}",
            session.getId(), sessions.size());
    }

    @Override
    protected void handleTextMessage(WebSocketSession session,
    TextMessage message) {
        String payload = message.getPayload();
        log.debug("Сообщение от {}: {}", session.getId(), payload);

        // Пример: echo-ответ или обработка команд
```



| WebSockets

```
// if ("PING".equals(payload)) { sendMessage(session, "PONG");
}

}

@Override
public void afterConnectionClosed(WebSocketSession session,
CloseStatus status) {
    sessions.remove(session);
    log.info("Отключение: id={}, причина={}, осталось: {}",
        session.getId(), status.getReason(), sessions.size());
}

@Override
public void handleTransportError(WebSocketSession session,
Throwable exception) {
    log.error("Ошибка транспорта для сессии {}: {}",
        session.getId(), exception.getMessage());
    sessions.remove(session);
}

/**
 * Рассылка сообщения всем подключенным клиентам.
 * Возвращает количество успешно отправленных сообщений.
 */
public int broadcast(String message) {
    TextMessage textMessage = new TextMessage(message);
    int sent = 0;

    for (WebSocketSession session : sessions) {
        if (sendMessage(session, textMessage)) {
            sent++;
        }
    }
}
```



| WebSockets

```
    }

    log.info("Broadcast:    отправлено    {}/{ }    клиентам",    sent,
sessions.size());
    return sent;
}

private boolean sendMessage(WebSocketSession session, TextMessage
message) {
    if (!session.isOpen()) {
        sessions.remove(session);
        return false;
    }
    try {
        // ConcurrentWebSocketSessionDecorator – альтернатива для
высоких нагрузок
        session.sendMessage(message);
        return true;
    } catch (IOException e) {
        log.warn("Ошибка отправки в сессию {}: {} ", session.getId(),
e.getMessage());
        sessions.remove(session);
        return false;
    }
}

/** Количество активных подключений (для мониторинга) */
public int getActiveConnections() {
    return sessions.size();
}
}
```



| WebSockets

Конфигурация (регистрация Endpoint)

Нам нужно сказать Spring'у: «если пришел запрос на /ws/notifications, отдай его нашему хендлеру».

```
import org.springframework.context.annotation.Configuration;
import
org.springframework.web.socket.config.annotation.EnableWebSocket;
import
org.springframework.web.socket.config.annotation.WebSocketConfigurer;
import
org.springframework.web.socket.config.annotation.WebSocketHandlerRegistry;

@Configuration
@EnableWebSocket
public class WebSocketConfig implements WebSocketConfigurer {

    private final NotificationWebSocketHandler notificationHandler;

    public          WebSocketConfig(NotificationWebSocketHandler
notificationHandler) {
        this.notificationHandler = notificationHandler;
    }

    @Override
    public void    registerWebSocketHandlers(WebSocketHandlerRegistry
registry) {
        registry.addHandler(notificationHandler, "/ws/notifications")
            // ВНИМАНИЕ: "*" только для разработки!
            //                                     В                продакшене:
        .setAllowedOrigins("https://yourdomain.com")
        .setAllowedOrigins("*");
    }
}
```



| WebSockets

```
// Для старых браузеров без WebSocket можно добавить SockJS
fallback:
    // .withSockJS();
}
}
```

Хендлер уже помечен @Component, поэтому Spring сам создаст бин. Мы просто инжектим его через конструктор — это современный подход (constructor injection).

Триггер событий (REST Controller)

Создадим контроллер, который будет имитировать событие системы.

```
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Map;

@RestController
@RequestMapping("/api/notifications")
public class NotificationController {

    private final NotificationWebSocketHandler handler;

    public NotificationController(NotificationWebSocketHandler handler)
    {
        this.handler = handler;
    }

    @PostMapping("/broadcast")
    public ResponseEntity<Map<String, Object>> broadcast(@RequestBody
String message) {
        int sent = handler.broadcast(message);
        return ResponseEntity.ok(Map.of(
            "status", "ok",
            "sentTo", sent,

```



| WebSockets

```
        "message", message
    ));
}

@GetMapping("/stats")
public ResponseEntity<Map<String, Object>> stats() {
    return ResponseEntity.ok(Map.of(
        "activeConnections", handler.getActiveConnections()
    ));
}
}
```

Клиентская часть

Создайте файл index.html в корне проекта (или просто на рабочем столе).

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>WebSocket demo</title>
    <style>
        body { font-family: sans-serif; padding: 20px; }
        #messages { border: 1px solid #ccc; padding: 10px; height: 300px;
overflow-y: scroll; }
        .log { color: gray; font-size: 0.8em; }
        .msg { color: blue; font-weight: bold; }
    </style>
</head>
<body>
    <h2>Live Notifications</h2>
    <div id="status">Status: Disconnected</div>
    <div id="messages"></div>

    <script>
```



| WebSockets

```
const statusDiv = document.getElementById('status');
const messagesDiv = document.getElementById('messages');

let socket = null;
let reconnectAttempts = 0;
let reconnectTimer = null;
const MAX_RECONNECT_DELAY = 30000; // макс. 30 сек между
попытками

function connect() {
    // Защита от повторного вызова
    if (socket && socket.readyState === WebSocket.CONNECTING) {
        return;
    }

    // Очищаем предыдущий таймер
    if (reconnectTimer) {
        clearTimeout(reconnectTimer);
        reconnectTimer = null;
    }

    // ws:// для http, wss:// для https (продакшен)
    const protocol = location.protocol === 'https:' ? 'wss:' :
'ws: ';
    socket = new
WebSocket(`${protocol}localhost:8080/ws/notifications`);

    socket.onopen = function() {
        reconnectAttempts = 0; // сброс счетчика при успешном
подключении

        statusDiv.textContent = 'Status: Connected';
        statusDiv.style.color = 'green';
    }
}
```



| WebSockets

```
        log('Соединение установлено');
    };

    socket.onmessage = function(event) {
        log('Получено: ' + event.data, 'msg');
    };

    socket.onclose = function(event) {
        statusDiv.textContent = 'Status: Disconnected';
        statusDiv.style.color = 'red';

        const reason = event.wasClean
            ? `закрито чисто, код=${event.code}`
            : 'обрыв соединения';
        log(`Отключение: ${reason}`);

        scheduleReconnect();
    };

    socket.onerror = function() {
        // WebSocket error event не содержит деталей (по
        // соображениям безопасности)
        log('Ошибка соединения');
    };
}

function scheduleReconnect() {
    // Exponential backoff: 1s, 2s, 4s, 8s... до
    // MAX_RECONNECT_DELAY
    const delay = Math.min(1000 * Math.pow(2,
    reconnectAttempts), MAX_RECONNECT_DELAY);
    reconnectAttempts++;
}
```



| WebSockets

```
        log(`Переподключение через ${delay/1000} сек (попытка  
${reconnectAttempts})...`);  
        reconnectTimer = setTimeout(connect, delay);  
    }  
  
    function log(text, type = 'log') {  
        const div = document.createElement('div');  
        div.className = type;  
        div.textContent = `[${new Date().toLocaleTimeString()}]  
${text}`;  
        messagesDiv.appendChild(div);  
        messagesDiv.scrollTop = messagesDiv.scrollHeight;  
    }  
  
    // Запуск  
    connect();  
</script>  
</body>  
</html>
```

Запуск и тест

1. Запустите Spring Boot приложение.
2. Откройте index.html в браузере. Вы должны увидеть «Status: Connected».
3. Откройте еще одну вкладку с этим файлом (симуляция второго пользователя).
4. Откройте Postman и отправьте уведомление

POST

Content-Type: text/plain

"Hello, WebSockets!"

http://localhost:8080/api/notifications/broadcast

Проверить статистику подключений:



WebSockets

GET

`http://localhost:8080/api/notifications/stats`

5. Посмотрите в обе вкладки браузера. Сообщение «Hello, WebSockets!» должно появиться мгновенно в обеих.

Live Notifications

Status: Connected

[13:16:27] Соединение установлено

[13:18:49] Получено: Content-Type: text/plain "Hello, WebSockets!"

Задания для самостоятельной работы

Задание 1: Ping/Pong (Keep-Alive)

- Цель: понять, как поддерживать соединение активным.
- Проблема: многие прокси-серверы (Nginx) и браузеры разрывают соединение, если по нему долго нет данных (таймаут неактивности).
- Задача:
 - на клиенте (JS) добавьте `setInterval`, который каждые 10 секунд отправляет сообщение «PING»;
 - на сервере (`handleTextMessage`) проверяйте: если пришел «PING», отправляйте обратно «PONG»;
 - убедитесь в логах сервера, что пинги приходят.

Задание 2: персональные уведомления

- Цель: научиться отправлять сообщения конкретному пользователю, а не всем.
- Задача:
 - измените URL подключения на клиенте, добавив ID пользователя: `ws://localhost:8080/ws/notifications?userId=user1`;
 - на сервере, в `afterConnectionEstablished`, извлеките query-параметры из `session.getUri().getQuery()`;



| WebSockets

- замените `List<WebSocketSession>` на `Map<String, WebSocketSession>`, где ключ — это `userId`;
- измените контроллер так, чтобы он принимал JSON `{"userId": "user1", "message": "Secret"}` и отправлял сообщение только в нужную сессию.

Дополнительные материалы

Полезные советы:

CORS: если ваш фронтенд лежит на `http://localhost:3000`, а бэкенд на 8080, браузер заблокирует сокет, если вы не добавите `.setAllowedOrigins("http://localhost:3000")` в конфиг. Для разработки можно ставить `*`, но в продакшене это дыра в безопасности.

WebSocket поддерживает `BinaryMessage`. Это полезно для передачи изображений или звука. В Spring для этого переопределяют метод `handleBinaryMessage`.

Хранить сессии в `List (in-memory)` работает только для одного экземпляра сервиса. Если у вас 2 инстанса сервиса, они ничего не знают о пользователях друг друга. Для решения этой проблемы нужен **Redis Pub/Sub** или **RabbitMQ**, чтобы синхронизировать события между инстансами (но это тема будущего занятия).

В чистых WebSockets нет встроенного понятия «авторизации» в момент рукопожатия так же удобно, как в HTTP. Часто токен авторизации (JWT) передают либо в query-параметре URL (что не очень безопасно, так как остается в логах), либо первым сообщением сразу после подключения. В более сложных системах (STOMP) это решается на уровне заголовков протокола.